

Busca Binária

- Imagine procurar “Kashiwabara” em uma lista impressa com 1 milhão de nomes.
- **Folha por folha?** Demoraria horas. Isso é **busca linear**: $\Theta(n)$.
- **Abrir no meio e decidir lado?** Em ~ 20 comparações achamos. Isso é **busca binária**: $\Theta(\lg n)$.

Pré-requisito crucial

A busca binária só funciona em arrays **ordenados**.

n	Linear (n)	Binária ($\lceil \lg n \rceil$)
100	100	7
1.000	1.000	10
1.000.000	1.000.000	20
1.000.000.000	1 bilhão	30

Reflexão: dobrar n adiciona apenas *uma* comparação na busca binária.

BINARY-SEARCH(A, v)

```
1   $low = 1$ 
2   $high = A.length$ 
3  while  $low \leq high$ 
4       $mid = \lfloor (low + high)/2 \rfloor$ 
5      if  $A[mid] == v$ 
6          return  $mid$ 
7      if  $A[mid] < v$ 
8           $low = mid + 1$ 
9      else  $high = mid - 1$ 
10 return NIL
```

$A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$, $v = 23$

Iter	low	high	mid	$A[mid]$ vs 23
1	1	10	5	$16 < 23 \Rightarrow \text{low}=6$
2	6	10	8	$56 > 23 \Rightarrow \text{high}=7$
3	6	7	6	$23 = 23 \Rightarrow$ retorna 6

A cada iteração o intervalo cai pela metade: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$. Após k iterações: $n/2^k \leq 1 \Rightarrow k \geq \lg n$.

Invariante

Se $v \in A$, então $v \in A[\text{low}..\text{high}]$.

Invariante de loop (while): Se v está em A , então $v \in A[low..high]$.

Inicialização: Antes da primeira iteração, $low = 1$ e $high = n$. Se $v \in A$, claramente $v \in A[1..n]$. ✓

Manutenção: Suponha o invariante verdadeiro no início de uma iteração.

- Se $A[mid] = v$, retornamos mid (correto).
- Se $A[mid] < v$: como A é ordenado, $v \notin A[low..mid]$. Logo, se $v \in A$, então $v \in A[mid + 1..high]$. Atualizamos $low = mid + 1$. ✓
- Se $A[mid] > v$: analogamente, $v \in A[low..mid - 1]$ e $high = mid - 1$. ✓

Terminação: Quando $low > high$, o intervalo é vazio. Pelo invariante, se v estivesse em A , estaria nesse intervalo vazio — contradição. Logo, $v \notin A$ e retornar NIL é correto. ✓

BINARY-SEARCH-RECURSIVE($A, v, low, high$)

```
1  if  $low > high$ 
2      return NIL
3   $mid = \lfloor (low + high) / 2 \rfloor$ 
4  if  $A[mid] == v$ 
5      return  $mid$ 
6  if  $A[mid] < v$ 
7      return BINARY-SEARCH-RECURSIVE( $A, v, mid + 1, high$ )
8  else return BINARY-SEARCH-RECURSIVE( $A, v, low, mid - 1$ )
```

Chamada inicial: BINARY-SEARCH-RECURSIVE($A, v, 1, A.length$)

- **Complexidade:** ambas $\Theta(\lg n)$ no tempo.
- **Espaço adicional:** iterativa usa $O(1)$; recursiva usa $O(\lg n)$ na pilha de chamadas.
- **Clareza:** a recursiva é mais declarativa e próxima da definição matemática.
- **Prática:** em geral, prefere-se a iterativa para evitar overhead de chamadas recursivas.

Relação de recorrência

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\lg n) \text{ (Teorema Mestre, caso 2).}$$

Exercício 1.1

Trace BINARY-SEARCH em $[1, 4, 7, 9, 11, 15, 20, 25, 30]$ procurando $v = 11$ e $v = 8$.

Exercício 1.2

Quantas comparações no *pior caso* para $n = 2048$?

Exercício 1.3

O que acontece se removermos o $\lfloor \cdot \rfloor$? Qual erro pode ocorrer?

Exercício 1.4

Modifique o algoritmo para retornar a posição de inserção quando $v \notin A$.

Insertion Sort

- Mão esquerda: cartas já **ordenadas**.
- Mão direita: pega próxima carta e insere no lugar certo.
- Após pegar todas, a mão esquerda está ordenada.

INSERTION-SORT(A)

```
1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3       $i = j - 1$ 
4      while  $i > 0$  e  $A[i] > key$ 
5           $A[i + 1] = A[i]$ 
6           $i = i - 1$ 
7       $A[i + 1] = key$ 
```

Traço completo: $A = [5, 2, 4, 6, 1, 3]$



j	Estado após inserir $A[j]$
2	[2, 5, 4, 6, 1, 3] – key=2 desloca 5
3	[2, 4, 5, 6, 1, 3] – key=4 desloca 5
4	[2, 4, 5, 6, 1, 3] – key=6 fica
5	[1, 2, 4, 5, 6, 3] – key=1 desloca tudo
6	[1, 2, 3, 4, 5, 6] – key=3

Melhor caso (já ordenado): while nunca executa $\Rightarrow \Theta(n)$.

Pior caso (decrecente): $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2} = \Theta(n^2)$.

Caso médio: $\Theta(n^2)$.

Espaço adicional: $O(1)$, in-place, **estável**.

Invariante de loop (for externo): No início da iteração j , o subarray $A[1..j - 1]$ contém os elementos originais de $A[1..j - 1]$, mas em ordem crescente.

Inicialização: Quando $j = 2$, o subarray $A[1..1]$ tem um único elemento, logo está trivialmente ordenado. ✓

Manutenção: Suponha $A[1..j - 1]$ ordenado. Pegamos $key = A[j]$ e o inserimos na posição correta em $A[1..j - 1]$:

- O loop while desloca elementos maiores que key para a direita.
- Ao final, inserimos key na posição correta.
- Portanto, $A[1..j]$ fica ordenado. ✓

Terminação: Quando $j = n + 1$, o invariante garante que $A[1..n]$ está ordenado. Como $n = A.length$, o array completo está ordenado. ✓

Exercício 2.1

Trace Insertion Sort em $[8, 3, 1, 7, 4, 2, 6, 5]$ mostrando o array após cada j .

Exercício 2.2

Conte exatamente quantas comparações e quantos deslocamentos ocorrem em $[5, 4, 3, 2, 1]$.

Exercício 2.3

Modifique para ordenar em ordem *decrecente*.

Exercício 2.4

Por que Insertion Sort é dito “online”? Dê um cenário real onde isso é útil.

Selection Sort

A cada passo: encontre o **mínimo** do trecho não ordenado e troque com a primeira posição não ordenada.

SELECTION-SORT(A)

```
1  for  $i = 1$  to  $n - 1$ 
2       $min = i$ 
3      for  $j = i + 1$  to  $n$ 
4          if  $A[j] < A[min]$ 
5               $min = j$ 
6      trocar  $A[i] \leftrightarrow A[min]$ 
```

Traço: $A = [5, 2, 4, 6, 1, 3]$



$i = 1: \text{min}=1 \Rightarrow [1, 2, 4, 6, 5, 3]$

$i = 2: \text{min}=2 \Rightarrow [1, 2, 4, 6, 5, 3]$

$i = 3: \text{min}=6 \Rightarrow [1, 2, 3, 6, 5, 4]$

$i = 4: \text{min}=6 \Rightarrow [1, 2, 3, 4, 5, 6]$

$i = 5: \text{min}=5 \Rightarrow [1, 2, 3, 4, 5, 6]$

Sempre $\Theta(n^2)$ comparações (independe da entrada). Apenas $O(n)$ trocas – útil quando *escrever* é caro (memória flash). **Não é estável**. In-place.

Invariante de loop (for externo): No início da iteração i , o subarray $A[1..i-1]$ contém os $(i-1)$ menores elementos de A em ordem crescente, e $A[i..n]$ contém os elementos restantes.

Inicialização: Quando $i = 1$, o subarray $A[1..0]$ é vazio, logo o invariante é trivialmente verdadeiro. ✓

Manutenção: Suponha o invariante válido para i . O loop interno encontra o índice min do menor elemento em $A[i..n]$. Ao trocar $A[i] \leftrightarrow A[min]$:

- $A[1..i]$ contém os i menores elementos em ordem crescente.
- $A[i+1..n]$ contém os elementos restantes. ✓

Terminação: Quando $i = n$, temos $A[1..n-1]$ com os $(n-1)$ menores elementos ordenados. O último elemento $A[n]$ é automaticamente o maior. Logo, $A[1..n]$ está ordenado. ✓

Exercício 3.1

Trace Selection Sort em $[4, 1, 3, 9, 7]$.

Exercício 3.2

Mostre por que Selection Sort *não* é estável usando $[2_a, 2_b, 1]$.

Exercício 3.3

Quantas trocas faz Selection Sort no pior caso? E no melhor?

Bubble Sort

Compare pares adjacentes; troque se fora de ordem. Repita.

BUBBLE-SORT(A)

```
1  for  $i = 1$  to  $n - 1$ 
2      for  $j = 1$  to  $n - i$ 
3          if  $A[j] > A[j + 1]$ 
4              trocar  $A[j] \leftrightarrow A[j + 1]$ 
```

Traço: $A = [5, 1, 4, 2, 8]$, passagem 1



$[5, 1, 4, 2, 8] \rightarrow [1, 5, 4, 2, 8] \rightarrow [1, 4, 5, 2, 8] \rightarrow [1, 4, 2, 5, 8] \rightarrow [1, 4, 2, 5, 8]$

O 8 já estava no lugar; o 5 “borbulhou” até a posição correta entre os não-ordenados.

Se uma passagem inteira não fizer trocas $\Rightarrow$ array já ordenado, pare. Melhor caso passa de $\Theta(n^2)$ para $\Theta(n)$.

Invariante de loop (for externo): No início da iteração i , os $i - 1$ maiores elementos estão nas posições $A[n - i + 2..n]$ em ordem crescente.

Inicialização: Quando $i = 1$, o subarray $A[n + 1..n]$ é vazio, logo o invariante é trivialmente verdadeiro. ✓

Manutenção: Suponha o invariante válido para i . O loop interno percorre $A[1..n - i]$ comparando pares adjacentes e trocando se necessário:

- O maior elemento de $A[1..n - i + 1]$ “borbulha” até a posição $n - i + 1$.
- Combinado com o invariante, temos os i maiores elementos em $A[n - i + 1..n]$ ordenados. ✓

Terminação: Quando $i = n$, os $(n - 1)$ maiores elementos estão em $A[2..n]$ ordenados. O primeiro elemento $A[1]$ é automaticamente o menor. Logo, $A[1..n]$ está ordenado. ✓

Exercício 4.1

Trace todas as passagens em $[3, 1, 4, 1, 5, 9, 2, 6]$.

Exercício 4.2

Implemente em pseudocódigo o Bubble Sort com flag de parada antecipada.

Exercício 4.3

O Bubble Sort é estável? Justifique.

Comparação e Síntese

Algoritmo	Melhor	Pior	Espaço adicional	Estável
Busca Binária	$\Theta(\lg n)$	$\Theta(\lg n)$	$O(1)$	N/A
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Sim
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Não
Bubble Sort*	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Sim

*com otimização de flag

Insertion Sort: arrays pequenos ($n < 50$) ou quase ordenados; processamento online.

Selection Sort: quando trocas são caras.

Bubble Sort: fim didático.

Arrays grandes: Merge Sort, Quick Sort, Heap Sort ($\Theta(n \lg n)$).

Exercício 5.1

Para cada algoritmo, identifique a entrada de tamanho 6 que produz o *pior* desempenho.

Exercício 5.2

Estabilidade: ordene $[(3, a), (1, b), (3, c), (2, d)]$ por chave usando cada algoritmo. Quais preservam a ordem a antes de c ?

Exercício 5.3

Mostre que se um array tem no máximo k inversões, Insertion Sort roda em $O(n + k)$.

Exercício 5.4

Desafio: prove (por invariante) a corretude da sua versão otimizada do Bubble Sort.

- Invariantes de loop = ferramenta para provar corretude.
- Mesmo $\Theta(n^2)$, os algoritmos têm *nuances* (estabilidade, trocas, melhor caso).
- Próxima aula: **Merge Sort** – como descer para $\Theta(n \lg n)$.